# Voyage-AI — Engineering Methodology & Operational Workflows

**Document Reference:** `docs/reports/methodology.pdf` / `docs/reports/methodology.md`
**System Version:** 0.2.1 (Phase 2 Verified Baseline)
**Author:** Senior Systems Analyst & QA Lead
**Verification Date:** October 25, 2026
**Status:** Implementation-Verified

---

## 1. Development Methodology & Engineering Lifecycle

### 1.1 Evidenced Engineering Process

The Voyage-AI engineering lifecycle follows a disciplined, evidence-based phased development model governed by pre-implementation architecture audits ( `docs/audit/PRE_IMPLEMENTATION_AUDIT_v1.1.md` ), a post-implementation verification report ( `docs/reports/POST_IMPLEMENTATION_VERIFICATION.md` ), and a formal implementation roadmap ( `docs/IMPLEMENTATION_PLAN.md` ).

The development methodology is characterized by three core tenets:
1. **Vertical Slice Verification Early:** Rather than authoring speculative backend layers or disconnected UI mockups, the engineering process requires building functional, testable vertical slices at each milestone.
2. **Offline-First Deterministic Testing:** Routine automated testing does not depend on uncontrolled external network services. External network boundaries are strictly encapsulated behind provider adapters with mock server fixtures, while live providers (Google Gemini, Open-Meteo, Wikimedia) are verified independently via dedicated test suites without mocks.
3. **Database Isolation & Safety Guarantees:** Test harnesses are programmatically prevented from contacting production or developer databases. Automated integration tests enforce execution against dedicated in-memory database instances ( `jdbc:h2:mem:voyage_flyway_test` ), while runtime operations target the single canonical MySQL database **`voyage`** on Docker port 3307.

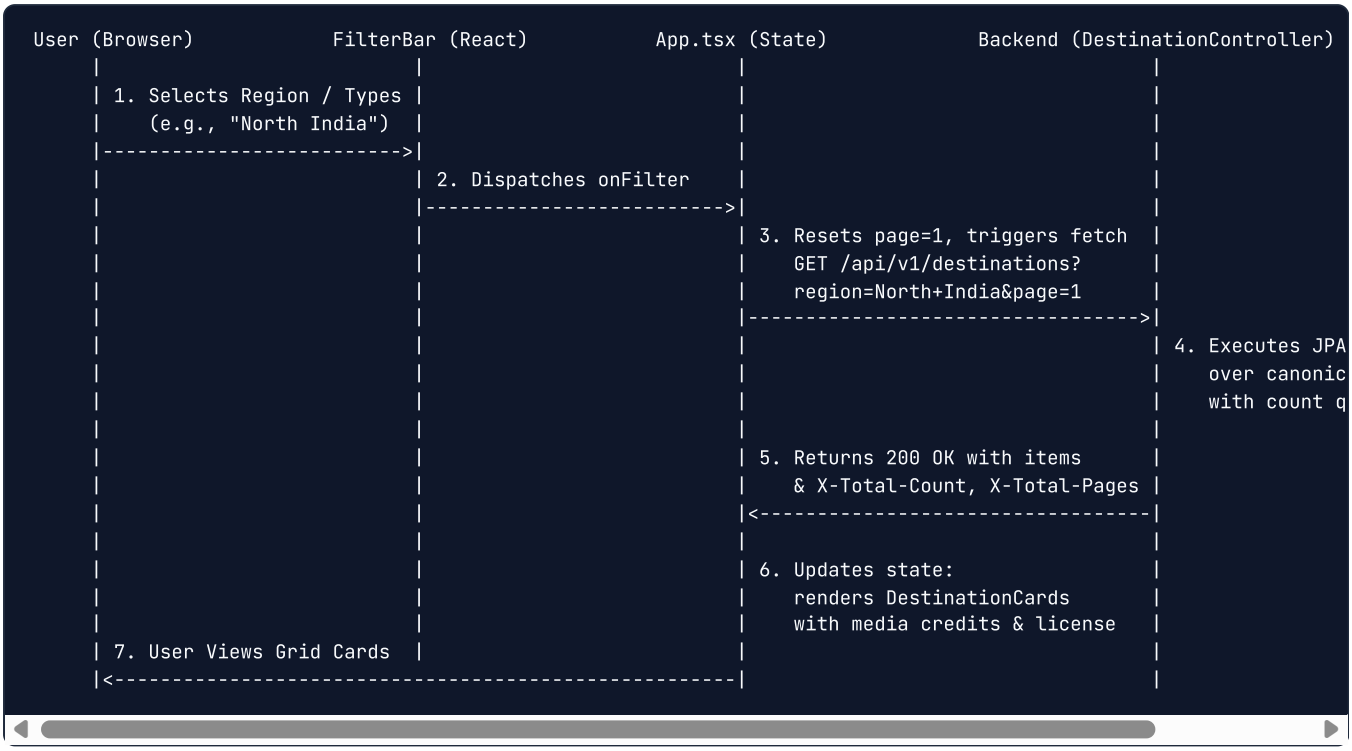
## 1.2 Module Implementation Sequence

```
[Phase 0: Foundation Setup] (COMPLETED & VERIFIED)
  |  ├── Repository scaffolding (frontend/, backend/, infra/, tests/)
  |  ├── Spring Boot 4.1.1 baseline & embedded Tomcat configuration (Port 8081)
  |  ├── Flyway migration baseline (V0__baseline.sql) & H2 test harness
  |  └── Custom health controller (/api/v1/health) and Spring Actuator (/actuator/health)
  v
[Phase 1: Design System & Discovery Prototype] (COMPLETED & VERIFIED)
  |  ├── Emil Kowalski design tokens in Tailwind CSS (warm modernism, tactile feedback)
  |  ├── Tactile component library (Button.tsx, Badge.tsx, SkeletonCard.tsx, EmptyState.tsx)
  |  ├── Responsive navigation and live health indicator (Navbar.tsx)
  |  ├── Curated Indian destination dataset (Jaipur, Varanasi, Munnar, Hampi)
  |  └── Multi-criteria discovery filtering & 5-tab detail modal dialog (DestinationDetailModal.tsx)
  v
[Phase 2: Relational Persistence, Security & Verified Integrations] (COMPLETED & VERIFIED)
  |  ├── Single Canonical MySQL 8.0 database (voyage) on port 3307; host port 3306 untouched
  |  ├── Flyway migrations V0-V4 (baseline, init schema, seeds, indexes, roles & licensing)
  |  ├── Domain JPA entities (User, Destination, TravelListing, Trip, TripDay, ItineraryItem)
  |  ├── Stateless JWT Bearer authentication filter chain & BCrypt password hashing
  |  ├── User profile persistence (/api/v1/users/me) and roles (ROLE_USER, ROLE_ADMIN)
  |  ├── Server-side catalog pagination (12 items/page) with full-catalog search and filter queries
  |  ├── Real Wikimedia ingestion pipeline with CC-BY-SA 4.0 / CC0 media licensing & attribution
  |  ├── Admin authorization enforcement (@PreAuthorize("hasRole('ADMIN')"))
  |  ├── User-scoped trip & itinerary CRUD with multi-tenant ownership isolation (403 Forbidden)
  |  ├── Live Google Gemini AI integration (verified with prompt "Capital of India" → "New Delhi")
  |  ├── Live Open-Meteo meteorological integration with coordinate caching & provenance
  |  ├── Windows start.bat development orchestrator (dependency order, health check polling, browser)
  |  └── Comprehensive test suite pass (128/128 tests verified passing, 100% pass rate)
  v
[Phase 3 Target: Interactive Leaflet Maps] (NEXT MILESTONE)
  |  ├── Leaflet.js map component with OpenStreetMap tiles
  |  └── Spatial destination & attraction plotting
  v
[Phase 4 Target: Transit Distance Routing]
  |  ├── OSRM transit matrix adapter with Haversine direct distance fallback
  └── Day schedule transit time estimation
```

---

## 2. End-to-End Operational Workflows

### 2.1 Workflow 1: Destination Discovery & Full-Catalog Paginated Filtering




### 2.2 Workflow 2: User Authentication & Profile Synchronization

1. **User Registration / Login:** The client dispatches credentials to `POST /api/v1/auth/register` or `POST /api/v1/auth/login`.
2. **Server Validation & Hashing:** The backend validates email format and password strength, verifies or hashes the password using BCrypt, and signs a stateless HMAC-SHA256 JWT containing `sub`, `email`, and `roles`.
3. **In-Memory Token Storage:** The client receives the JWT and stores it strictly in React application memory—never in `localStorage` or unencrypted cookies.
4. **Profile Management:** The client accesses `GET /api/v1/users/me` and updates preferences via `PUT /api/v1/users/me`. Server-side validation prevents non-admin users from elevating their role to `ROLE_ADMIN`.

### 2.3 Workflow 3: Multi-Day Trip Planning & Ownership Enforcement

1. **Trip Initialization:** Authenticated user submits `POST /api/v1/trips` with title, destination, start date, end date, and budget.
2. **Date Range Validation & Day Generation:** The backend verifies `startDate ⩽ endDate` (returning HTTP 400 Bad Request on inverted dates) and automatically generates `TripDay` records for each calendar day in the interval.
3. **Multi-Tenant Ownership Check:** Every trip mutation or read endpoint (`GET /api/v1/trips/{id}`, `PUT`, `DELETE`, `POST /api/v1/trips/{id}/days`) verifies that `trip.user.id == authenticatedPrincipal.id`. Unauthorized cross-user access immediately returns `403 Forbidden`.

### 2.4 Workflow 4: Real Wikimedia Ingestion Pipeline with Creative Commons Attribution

1. **Ingestion Request:** An authorized administrator ( `ROLE_ADMIN` ) issues `POST /api/v1/admin/destinations/import` specifying candidate destination names.
2. **Summary Extraction:** `DestinationIngestionService` queries the English Wikipedia REST summary API ( `https://en.wikipedia.org/api/rest_v1/page/summary/{title}` ).
3. **Media & License Audit:** For each destination image, the service queries the MediaWiki Action API ( `https://commons.wikimedia.org/w/api.php?action=query&prop=imageinfo&iiprop=extmetadata|user|url` ) to extract the artist/photographer name, explicit license short name ( `CC-BY-SA 4.0` , `CC-BY 2.0` , `CC0` , `Public Domain` ), and official license URL.
4. **Non-Fabricated Persistence:** Records are persisted to `voyage.destinations` with non-nullable provenance columns, `image_credit` , and `image_license_url` . Destinations without curated plans default to `recommendedDurationDays = 0` and `curationStatus = "COMMUNITY_INGESTED"` .

### 2.5 Workflow 5: Grounded AI Reasoning & Weather Forecast Retrieval

1. **AI Travel Assistant:** User submits a natural language travel inquiry in `AiTravelAssistant.tsx` .
2. **Grounded Prompting:** The backend frames the prompt strictly around verified destination entities and zero user PII, transmitting it to Google Gemini ( `gemini-3.5-flash-lite` ). Upstream responses are formatted with markdown and rendered in the chat UI.
3. **Live Meteorological Forecast:** The client requests `GET /api/v1/weather?latitude=...&longitude=...` . `OpenMeteoWeatherService` queries Open-Meteo's forecast API, maps WMO weather codes, and returns verified metrics with CC-BY 4.0 attribution.

---

## 3. Automated Testing & Quality Methodology

### 3.1 Multi-Tier Test Execution Strategy

The verification methodology guarantees 100% deterministic reproducibility across three isolated tiers:

```
================================================================================================
Testing Tier              Framework / Tool             Tests   Passed  Failed   Skipped
================================================================================================
Backend Unit & Domain     JUnit 5, MockMvc, Mockito       83       83       0         0
Frontend Component Unit   Vitest, React Testing Library   11       11       0         0
Browser & Black-Box API   Java Playwright, HttpClient     34       34       0         0
------------------------------------------------------------------------------------------------
TOTAL VOYAGE-AI SUITE                                     128      128       0         0
================================================================================================
Pass Rate: 100.0%
```

### 3.2 Database Safety Guarantees

All automated unit and controller tests execute against transient in-memory H2 instances ( `jdbc:h2:mem:voyage_flyway_test;MODE=MySQL` ). `DatabaseFlywayIntegrationTest.java` enforces programmatic assertions verifying that `url` never contains `localhost:3306/voyage` , `localhost:3306/pat` , or `:3307` . The developer's host database is guaranteed zero modification during test execution.

---

## 4. Local Development Orchestration ( `start.bat` )

To eliminate setup friction, Voyage-AI provides `start.bat` in the repository root:
1. **Verification & Environment Check:** Detects `.env` and provisions from `.env.example` if missing.
2. **MySQL Database Check:** Probes port `3307` (Docker canonical MySQL). If down, probes port `3306`. If neither is listening, invokes `docker compose up -d mysql` and polls up to 30s.
3. **Spring Boot Backend Check:** Probes `http://localhost:8081/api/v1/health`. If down, attempts Docker container launch or falls back to local `mvn spring-boot:run`.
4. **Vite Frontend Check:** Probes `http://localhost:5173`. If down, checks `node_modules` (installs if missing) and launches `npm run dev`.
5. **Browser Launch:** Displays active service URLs and opens `http://localhost:5173` in the default web browser.
6. **Non-Destructive Guarantee:** Reuses already-running services, preserves existing ports, and never resets or deletes database records.